🧠 Fine-Tuning — Cheat Sheet

Suresh D R | AI Product Developer & Technology Mentor

---

1. What Is Fine-Tuning?

Taking a pre-trained LLM and continuing to train it on your own data so it becomes a specialist in your domain. The model already knows language — you teach it YOUR company's facts, tone, and terminology.

```
Pre-trained model  →  knows general language, knows nothing about YOU
Fine-tuned model   →  knows general language  +  knows YOUR domain ✅
```

---

2. Fine-Tuning vs RAG vs Both

Situation	Use
Documents change frequently	RAG
Need to cite sources	RAG
Need specific tone / style	Fine-Tuning
Need consistent answers	Fine-Tuning
Best production setup	Fine-Tuning + RAG together

---

3. Types of Fine-Tuning

Type	Labelled Data?	What You Need	Best For
Self-Supervised	❌ NO	Raw documents only	Domain immersion, terminology
Supervised (SFT)	✅ YES	500+ Q&A pairs	Exact Q→A behaviour
Instruction FT	✅ YES	(instruction, input, output) triples	Tone, format, role
RLHF	✅ YES (human rankings)	Millions in human annotation	ChatGPT / Claude quality

> **LoRA and QLoRA are METHODS, not data types.** Use them with any type above.

---

4. Self-Supervised — How Next-Word Prediction Works

```
Document text: "Maternity leave at TechNova is 26 weeks, fully paid."

Auto-generated training examples (no labels written by hand):

  Input: "Maternity leave at TechNova is"        → Target: "26"
  Input: "Maternity leave at TechNova is 26"     → Target: "weeks"
  Input: "Maternity leave at TechNova is 26 weeks,"  → Target: "fully"

One sentence = multiple training examples.
25,000-word document = ~25,000 examples automatically. ✅
```

---

5. PEFT — Parameter-Efficient Fine-Tuning

```
PEFT
  ├── LoRA   - add small adapter matrices A and B alongside frozen weights
  ├── QLoRA  - LoRA + 4-bit quantised base model (8× less memory)
  ├── Prefix Tuning  - trainable virtual tokens prepended to input
  └── Prompt Tuning  - soft embeddings only (very few params, lower quality)

Industry standard today: LoRA and QLoRA
```

6. LoRA — Key Formula and Calculations

```
Original weight matrix W:   FROZEN — never updated

LoRA adds two small matrices:
  Matrix A:  (d_model × r)   ← trained
  Matrix B:  (r × d_model)   ← trained

Forward pass:
  output = x × W  +  (x × A × B) × (alpha / r)
           ↑ frozen     ↑ adapter learns the delta
```

Parameter Calculation — LLaMA 3 8B, r = 16

```
Original Wq layer:   4096 × 4096  =  16,777,216 parameters
LoRA Matrix A:       4096 × 16    =      65,536 parameters  ← trained
LoRA Matrix B:         16 × 4096  =      65,536 parameters  ← trained
Total LoRA params:                =     131,072 parameters

Reduction:  131,072 ÷ 16,777,216  =  0.78% of original  🎯

Across all 32 layers (Wq + Wv):
  64 adapters × 131,072  =  8,388,608 trainable params
  Total model params     =  8,030,261,248
  Trainable %            =  0.104%   ← you train only 0.1%!
```

Where Does 4096 × 4096 Come From?

```
LLaMA 3 8B design choice:  hidden dimension = 4096
Every token is represented as a vector of 4096 numbers.
The weight matrix transforms 4096 → 4096, so shape = (4096 × 4096).
The designer chose 4096. Different models choose different sizes:
  GPT-2      →  768 × 768
  LLaMA 3 8B →  4096 × 4096
  LLaMA 3 70B → 8192 × 8192
```

Alpha (α) — What It Controls

```
Formula:  output = x×W  +  (x × A × B) × (alpha / r)

alpha controls HOW STRONGLY the adapter influences the output.
The ratio alpha/r is what matters:

  r=8,  alpha=8   → ratio = 1.0  (conservative, balanced)
  r=8,  alpha=16  → ratio = 2.0  (moderate — most common)
  r=16, alpha=32  → ratio = 2.0  (more capacity, same strength)

alpha too high → adapter dominates, model loses general ability ⚠️
alpha too low  → adapter barely contributes, fine-tuning has no effect ⚠️
Rule: start with alpha = 2 × r
```

Initialisation

```
A ← random Gaussian   (small nonzero values)
B ← all zeros

At step 0:  B × A = 0  →  adapter contributes nothing
Model = identical to base model at the start.
Adapters gradually learn as training progresses. ✅
```

LoRA Rank r — Which to Choose

r	Params per matrix	Use Case
4	65,536	Simple style/tone changes
8	131,072	Standard — best starting point
16	262,144	Complex domain adaptation
32	524,288	High quality, approaches full FT
64	1,048,576	Very close to full FT quality

Which Layers to Apply LoRA To

Layer	Apply?	Why
Wq — Query	✅ YES (default)	Most impactful for domain adaptation
Wv — Value	✅ YES (default)	Second most impactful
Wk — Key	Sometimes	Adds capacity for very different domains
Wo — Output	Sometimes	Marginal gains
Feed-forward	Aggressive only	Very different domains (code, science)

---

7. QLoRA — Quantisation + LoRA

What Are Bits?

```
A bit stores 0 or 1. More bits = more precision = more memory.

  float32  →  32 bits  =  4 bytes per weight   (full precision)
  float16  →  16 bits  =  2 bytes per weight   (2× smaller)
  int8     →   8 bits  =  1 byte  per weight   (4× smaller)
  NF4      →   4 bits  =  0.5 bytes per weight (8× smaller) ← QLoRA
```

Memory Savings

```
Model: LLaMA 3 7B (7,000,000,000 parameters)

  float32  :  7B × 4.0 bytes  =  28.0 GB
  float16  :  7B × 2.0 bytes  =  14.0 GB
  int8     :  7B × 1.0 bytes  =   7.0 GB
  NF4 4-bit:  7B × 0.5 bytes  =   3.5 GB  ← fits on 8 GB GPU! ✅
```

NF4 Quantisation — How One Weight Is Stored

```
Original value (float32):  0.3847  (4 bytes = 32 bits)

NF4 has 16 fixed values (from normal distribution quantiles):
  Index:  0      1       2       3  ...  11      12    ...
  Value: -1.0  -0.6962 -0.5251 -0.3949  0.3949  0.5251

Step 1: find nearest NF4 value → 0.3949  (index 11)
Step 2: store index 11 = binary 1011     (only 4 bits!)
Step 3: dequantise → read 1011 → 11 → 0.3949

Error:  |0.3949 - 0.3847|  =  0.0102  (tiny, negligible) ✅
Saving: 32 bits → 4 bits  =  8× less memory
```

QLoRA Full Memory Budget — LLaMA 3 8B

```
Base model (4-bit NF4, frozen):  8B × 0.5 bytes =   4.0 GB
LoRA adapters (float32):         8.4M × 4 bytes =    33 MB
Gradients (adapters only):                      =    33 MB
Optimizer (adapters only):                      =    67 MB
KV Cache (estimate):                            =   500 MB
TOTAL:                                          ≈  4.6 GB

Compare: full fine-tuning = 128 GB needed
QLoRA runs on a free Colab T4 GPU (15 GB)! ✅
```

---

8. Training Parameters — Quick Reference

Parameter	What It Does	Typical Value
`num_train_epochs`	How many times model sees full dataset	3–5
`learning_rate`	Step size for weight updates	`2e-4` for LoRA
`per_device_train_batch_size`	Examples processed per step	2–4 (consumer GPU)
`gradient_accumulation_steps`	Accumulate N batches before update	2–4

Parameter	What It Does	Typical Value
`warmup_ratio`	Fraction of steps to warm up LR	0.03
`weight_decay`	Regularisation — prevents overfitting	0.01
`lora_dropout`	Randomly zero adapter activations	0.05–0.1
`lora_alpha`	Adapter strength scaling	2 × r
`r` (rank)	Adapter size	8 or 16
`logging_steps`	Print loss every N steps	5–10

What Is a Step?

```
One step = one weight update = model processes one batch and adjusts weights.

Total steps = (training examples ÷ batch_size) × num_epochs

Example:
  80 examples,  batch_size = 4,  epochs = 8
  Steps per epoch = 80 ÷ 4  = 20
  Total steps     = 20 × 8  = 160

  Step  5 | Loss: 2.84  ← model knows nothing yet
  Step 40 | Loss: 1.52  ← learning
  Step 160 | Loss: 0.43 ← well trained

Loss going DOWN    → model is learning              ✅
Loss going UP      → overfitting, stop now          ⚠️
Loss not moving    → learning rate too low          ⚠️
Loss jumping around → learning rate too high        ⚠️
```

Learning Rate Notation

```
2e-4  =  2 × 10⁻⁴  =  0.0002
1e-5  =  1 × 10⁻⁵  =  0.00001
3e-5  =  3 × 10⁻⁵  =  0.00003
```

9. Layer-by-Layer — What Gets Updated

```
Transformer Layer (LoRA typical setup):

  Layer Norm               → FROZEN
  Wq  (Query)              → FROZEN  +  LoRA A, B  ← trained
  Wk  (Key)                → FROZEN
  Wv  (Value)              → FROZEN  +  LoRA A, B  ← trained
  Wo  (Output projection)  → FROZEN
  Feed-Forward W_up        → FROZEN
  Feed-Forward W_gate      → FROZEN
  Feed-Forward W_down      → FROZEN

Base model knowledge  →  preserved in frozen weights
New domain knowledge  →  stored in small adapter matrices
```

10. Common Problems and Fixes

Problem	Signs	Fix
Catastrophic Forgetting	Model forgets general tasks	Use LoRA (base frozen); mix 10–20% general data
Overfitting	Train loss → 0, test loss high	Add `lora_dropout=0.1` ; fewer epochs; more data
Mode Collapse	Same answer every question	Diversify training examples
Out of Memory	CUDA OOM error	Reduce batch size; increase gradient_accumulation_steps; switch to QLoRA
No improvement	Loss barely moves	Increase learning rate; increase r; more epochs

11. After Training — Deployment Options

```
Option A — Keep adapter separate:
  Load base model  +  load adapter on top
  Slight inference overhead
  Useful for hot-swapping multiple adapters

Option B — Merge into base (most common):
  W_final = W + (B × A) × (alpha / r)
  Zero inference overhead
  Cannot separate after merging

  model = model.merge_and_unload()
  model.save_pretrained("./my-finetuned-model")
```

---

12. LoRA Code — Quick Reference

```python
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
    r              = 16,          # rank
    lora_alpha     = 32,          # scaling (ratio = alpha/r = 2.0)
    target_modules = ["q_proj", "v_proj"],  # which layers
    lora_dropout   = 0.05,
    task_type      = "CAUSAL_LM"
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable params: 4,194,304 || all params: 8,030,261,248
# trainable%: 0.052%
```

QLoRA Code — Quick Reference

```python
from transformers import BitsAndBytesConfig
import torch

bnb_config = BitsAndBytesConfig(
```

```
    load_in_4bit              = True,
    bnb_4bit_quant_type       = "nf4",
    bnb_4bit_compute_dtype    = torch.float16,
    bnb_4bit_use_double_quant = True
)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3-8b",
    quantization_config = bnb_config,
    device_map          = "auto"
)
```

---

13. Decision Guide — When to Use What

```
GPU memory?
  < 8 GB    → QLoRA, r=8
  8–24 GB   → QLoRA, r=16
  24–80 GB  → LoRA,  r=16 or r=32
  > 80 GB   → Full fine-tuning (rare)

Have labelled data?
  NO   → Self-supervised FT on raw documents (no labels needed)
  YES  → Supervised FT (Q&A pairs) or Instruction FT

Domain difference from general text?
  Slight (new terminology)     → r=8,  alpha=16
  Moderate (technical jargon)  → r=16, alpha=32
  High (code, formulas)        → r=64, alpha=64

Dataset size?
  < 1,000 examples  → r=4 or r=8,  1–3 epochs
  1,000–10,000      → r=16,         3–5 epochs
  > 10,000          → r=32+,        5–10 epochs
```

---

14. Key Numbers to Remember

```
LoRA r=16 on LLaMA 3 8B:
  Trainable params     :  8,388,608   (0.1% of 8B total)
  Reduction per matrix :  131,072 ÷ 16,777,216  =  0.78%
  Adapter file size    :  ~33 MB
  Training time A100   :  hours, not days

QLoRA 4-bit on 7B model:
  GPU memory needed    :  ~4.6 GB   (vs 128 GB full FT)
  Memory saving        :  8× reduction from float32
  Quality vs full FT   :  95–98%
  Runs on              :  free Colab T4 (15 GB)

Self-supervised on 25,000-word doc:
  Training examples    :  ~25,000  (auto-created, no labels)
  Training time        :  30–60 min on one GPU
  Cloud cost           :  under $5

Auto-generate SFT labels with GPT:
  100 chunks × 5 pairs :  500 training examples
  API cost             :  ~$2
  Time                 :  5–10 minutes
```
